

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

//--- Function Prototypes ---
void display(int arr[], int size);
void swap(int *a, int *b);
void bubblesort(int arr[], int size);
void selectionsort(int arr[], int size);
void insertionsort(int arr[], int size);

//--- Main Program Logic ---
int main() {
    int n = 0, option;

    // Input validation for array size
    while (n <= 0) {
        printf("Enter the number of elements for the array (must be > 0): ");
        scanf("%d", &n);
        if (n <= 0) {
            printf("Invalid size. Please try again.\n");
        }
    }

    int original_arr[n];
    int arr_to_sort[n];

    printf("\nEnter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &original_arr[i]);
    }

    clock_t start, end;
    double cpu_time_used;

    while (1) {
        printf("\n--- Sorting Menu ---\n");
        printf("1. Bubble Sort\n");
        printf("2. Selection Sort\n");
        printf("3. Insertion Sort\n");
```

```

printf("4. Exit\n");
printf("Choose an option: ");
scanf("%d", &option);

// Before every sort, create a fresh copy of the original array
for(int i = 0; i < n; i++) {
    arr_to_sort[i] = original_arr[i];
}

if (option >= 1 && option <= 3) {
    printf("\nOriginal array: ");
    display(original_arr, n);
}

switch (option) {
    case 1:
        start = clock();
        bubblesort(arr_to_sort, n);
        end = clock();
        cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
        printf("Array sorted with Bubble Sort: ");
        display(arr_to_sort, n);
        printf("🕒 Bubble Sort took %f seconds.\n", cpu_time_used);
        break;

    case 2:
        start = clock();
        selectionsort(arr_to_sort, n);
        end = clock();
        cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
        printf("Array sorted with Selection Sort: ");
        display(arr_to_sort, n);
        printf("🕒 Selection Sort took %f seconds.\n", cpu_time_used);
        break;

    case 3:
        start = clock();
        insertionsort(arr_to_sort, n);
        end = clock();
        cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
        printf("Array sorted with Insertion Sort: ");
        display(arr_to_sort, n);
        printf("🕒 Insertion Sort took %f seconds.\n", cpu_time_used);

```

```

        break;

    case 4:
        printf("Exiting program. Goodbye!\n");
        exit(0);

    default:
        printf("Invalid option! Please choose between 1 and 4.\n");
    }
}

return 0;
}

```

//--- Function Definitions ---

```

void display(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

```

```

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

```

```

void bubblesort(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(&arr[j], &arr[j + 1]);
            }
        }
    }
}

```

```

void selectionsort(int arr[], int size) {
    int i, j, min_index;
    for (i = 0; i < size - 1; i++) {
        min_index = i;

```

```

        for (j = i + 1; j < size; j++) {
            if (arr[j] < arr[min_index]) {
                min_index = j;
            }
        }
        if (min_index != i) {
            swap(&arr[i], &arr[min_index]);
        }
    }
}

```

```

void insertionsort(int arr[], int size) {
    int i, key, j;
    for (i = 1; i < size; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

```

Sample Execution Output

Enter the number of elements for the array (must be > 0): 6

Enter 6 elements:

88

19

42

7

91

34

--- Sorting Menu ---

1. Bubble Sort

2. Selection Sort

3. Insertion Sort

4. Exit

Choose an option: 1

Original array: 88 19 42 7 91 34

Array sorted with Bubble Sort: 7 19 34 42 88 91

 Bubble Sort took 0.000002 seconds.

--- Sorting Menu ---

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Exit

Choose an option: 2

Original array: 88 19 42 7 91 34

Array sorted with Selection Sort: 7 19 34 42 88 91

 Selection Sort took 0.000001 seconds.

--- Sorting Menu ---

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Exit

Choose an option: 3

Original array: 88 19 42 7 91 34

Array sorted with Insertion Sort: 7 19 34 42 88 91

 Insertion Sort took 0.000001 seconds.

--- Sorting Menu ---

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Exit

Choose an option: 4

Exiting program. Goodbye!

Algorithms and Analysis

Overall Program Algorithm

This describes the flow of the main application.

1. **Start** the program.
2. **Initialize Variables:** Declare variables for array size (n), user option (option), and timing.
3. **Input Array Size:** Prompt the user to enter the number of elements for the array.
4. **Validate Input:** Loop until the user enters a size greater than 0.
5. **Declare Arrays:** Create two integer arrays of size n: original_arr and arr_to_sort.
6. **Input Array Elements:** Prompt the user to enter the elements and store them in original_arr.
7. **Start Main Loop:** Begin an infinite while loop to display the menu.
8. **Display Menu:** Show the user the options: 1 for Bubble Sort, 2 for Selection Sort, 3 for Insertion Sort, and 4 to Exit.
9. **Get User Choice:** Read the integer option chosen by the user.
10. **Create Array Copy:** Before performing any sort, copy all elements from original_arr to arr_to_sort. This ensures each sort works on the same initial data.
11. **Process Choice (Switch Statement):**
 - **Case 1, 2, or 3:**
 - Record the start time using clock().
 - Call the corresponding sort function (bubblesort, selectionsort, or insertionsort) with arr_to_sort.
 - Record the end time using clock().
 - Calculate the cpu_time_used in seconds.
 - Print the sorted array and the time taken.
 - **Case 4:** Print an exit message and terminate the program using exit(0).
 - **Default:** If the choice is invalid, print an error message.
12. **Loop Continuation:** The loop automatically repeats from Step 8.

Bubble Sort

Algorithm:

1. Start from the beginning of the array (index 0).
2. Compare the first element with the second. If the first is greater than the second, swap them.
3. Move to the next pair of elements and repeat the comparison and swap if necessary.
4. Continue this process until the end of the array is reached. This completes the first pass and places the largest element at the end.
5. Repeat the entire process for the remaining n-1 elements. Each pass will place the next

largest element in its correct position. The total number of passes required is $n-1$.

Complexity Analysis:

- **Time Complexity:**
 - **Worst Case:** $O(n^2)$ (when the array is in reverse order).
 - **Average Case:** $O(n^2)$
 - **Best Case:** $O(n)$ (when the array is already sorted, with an optimized implementation).
- **Space Complexity:** $O(1)$ (in-place algorithm).

Selection Sort

Algorithm:

1. Set the first element as the minimum ($\text{min_index} = 0$).
2. Iterate through the rest of the unsorted array (from the second element to the end).
3. In each iteration, if an element is found to be smaller than the current minimum, update min_index to the index of that element.
4. After scanning the entire unsorted part, if the min_index is different from the starting index, swap the two elements.
5. Increment the boundary of the sorted sub-array by one and repeat the process for the new unsorted part.

Complexity Analysis:

- **Time Complexity:**
 - **Worst Case:** $O(n^2)$
 - **Average Case:** $O(n^2)$
 - **Best Case:** $O(n^2)$ (The number of comparisons is always the same).
- **Space Complexity:** $O(1)$ (in-place algorithm).

Insertion Sort

Algorithm:

1. Start from the second element (index 1), assuming the first element is a sorted sub-array of size one.
2. Select the current element and store it in a key variable.
3. Compare the key with the elements in the sorted sub-array, moving from right to left.
4. Shift all elements in the sorted sub-array that are greater than the key one position to the right.
5. Insert the key into the correct position (the empty slot created by the shifting).
6. Repeat this process for all remaining elements in the unsorted part of the array.

Complexity Analysis:

- **Time Complexity:**
 - **Worst Case:** $O(n^2)$ (when the array is in reverse order).

- **Average Case:** $O(n^2)$
- **Best Case:** $O(n)$ (when the array is already sorted).
- **Space Complexity:** $O(1)$ (in-place algorithm).